MAS 架构模式报告

—— 从多智能体系统工程视角重新审视 28 种治理制度

核心思路

不再按历史分类。只问一个问题：**如果我要用代码实现这些系统，它们的消息流拓扑到底有几种?**

从 MAS 构建角度，真正决定架构差异的只有三个维度：

1. **消息流模式**：线性、分支、循环、广播、共识
2. **控制模式**：谁能阻断流程？谁有否决权？
3. **User 接入点**：任务从哪里进入系统？结果从哪里返回？

第一部分：六种 MAS 架构模式

经过对 28 种制度的拓扑抽象，所有制度可以归结为 **6 种 MAS 架构模式**。

模式 1：单链管道（Simple Pipeline）

```
User ──→ Planner ──→ [Manager] ──→ Executor ──→ Output
            │              │
            │          (可选分支)
            │              │
            ▼              ▼
        Manager-1      Manager-2
            │              │
            ▼              ▼
        Executor       Executor
```

消息流： 单向，从 User 到 Executor，无回路。中间可以有扇出（一对多分发），但没有任何节点能让消息"往回走"。

User 位置： 系统入口，第一个节点。User 发出 Prompt，等最终 Output。中间过程 User 不介入。

控制逻辑： 无。没有任何节点可以 reject、veto 或中断流程。消息只能向前流。

对应制度：

制度	备注
古埃及法老制	最纯粹的单链
罗马天主教会圣统制	有领域分支（Dicastery），但无回路
法国绝对君主制	Reviewer 节点存在但 disabled，等价于无
罗马帝国元首制	元老院虚化，等价于无 Reviewer
明清内阁制	内阁票拟 → 皇帝朱批 → 执行。看似有审核，但"朱批"只是 User 自己确认自己，不是独立 Reviewer
苏联体制	最高苏维埃虚化，Gosplan 只做资源调度不做逻辑审核
哈里发制	卡迪的"合规判定"是 Executor 自带的，不是独立 Reviewer 节点
奥斯曼御前会议制	大维齐尔是全权代理 Planner，文书官是 Formatter，无独立审核

总计：8 个制度

关键洞察： 从 MAS 角度看，一个 Reviewer 节点如果没有 `reject` 权限，或者 `enabled=false`，或者只做日志记录，它就**不存在**。不要被历史名称迷惑——元老院、最高苏维埃、三级会议在代码层面都是注释掉的死代码。

明清的特殊性： 皇帝朱批看似是"审核"，但那是 User 审核 Planner 的输出，然后决定是否启动执行。这在 MAS 里叫 **Human-in-the-loop confirmation**，不是独立 Reviewer。等价于 User 发 Prompt → Planner 返回 Plan → User 确认 "OK，执行" → Executor 开始。这仍然是单链管道，只是 User 在链条中间插了一个确认步骤。

奥斯曼的特殊性： 苏丹把全部权力委托给大维齐尔。在 MAS 中这等价于 `User.delegate(planner)`——User 只在最后验收，中间完全不介入。御前会议（Divan）是多专家协作，但它是 Planner 内部的实现细节（Mixture of Experts），不是独立的拓扑节点。

模式 2：带审核回路的管道（Pipeline with Review Loop）

```
User ──→ Planner ──→ Reviewer ──→ Coordinator ──→ Executor ──→ Output
               ▲           │
               │   Reject  │
               └───────────┘
```

消息流： 基本是单向管道，但在 Planner 和 Executor 之间插入了一个 **Reviewer 节点**，它有权力将方案打回 Planner 重写。这形成了一个**局部回路（loop）**。

User 位置： 仍然是系统入口。User 发出初始需求，但日常不介入 Plan-Review 循环。只在最终验收时出现。

控制逻辑： Reviewer 拥有 `reject` 权限。如果方案不合格，消息不会继续往下流，而是回到 Planner。这是整个架构中唯一的"回路"。

对应制度：

制度	Reviewer 是谁	回路机制
唐代三省六部	门下省	封驳 → 返回中书省重写

总计：1 个制度（唐代三省六部是这个模式的唯一纯粹代表）

为什么只有一个？ 因为"有一个独立节点，专门审核 Planner 的输出，并且有权打回重写"这个设计在历史上非常罕见。大多数制度要么没有审核（模式 1），要么把审核放在执行之后而非之前（模式 3），要么把审核分散到多个节点（模式 5）。

MAS 实现要点：

- 需要定义 `max_review_rounds`（最大审核轮数），防止 Planner 和 Reviewer 无限循环
- Reviewer 的 System Prompt 需要和 Planner 不同（如果用同一个模型，审核就没意义）
- 这个模式天然适合 **Self-Consistency / Reflection** 范式

模式 3：带旁路监控的管道（Pipeline with Parallel Monitor）

```
User ◀──── 异常报告 ──── Monitor (Read-only)
  │                          ▲
  ▼                          │ 实时采样
Planner ──→ Manager ──→ Executor ──→ Output
```

消息流： 主链路仍然是单向管道。但存在一个**独立的 Monitor 节点**，它不在主链路上，而是"旁路"挂载在 Manager/Executor 旁边，以 Read-only 权限采样它们的行为，如果检测到异常则直接

向 User 报告。

User 位置： 系统入口 + 异常报告的接收端。User 同时接收两条信息流：正常的执行结果（从 Executor）和异常报告（从 Monitor）。

控制逻辑： Monitor **不阻断主流程**。它不能 reject 或 veto。它只是"看"，然后把问题报给 User，由 User 决定是否人工干预。

与模式 2 的本质区别：

- 模式 2（审核回路）：错误在**执行前**被拦截，代价是增加延迟
- 模式 3（旁路监控）：错误在**执行中/后**被发现，不增加延迟，但发现时可能已经造成损失

对应制度：

制度	Monitor 是谁	监控对象
波斯行省制	"国王之眼"（王室监察官）	总督/萨特拉普
秦汉郡县制	御史大夫	丞相/太尉及地方官

总计：2 个制度

MAS 实现要点：

- Monitor 是一个独立进程/线程，异步运行
- Monitor 需要 Read-only 权限访问 Manager/Executor 的中间状态
- Monitor 的输出不进入主消息流，而是通过独立通道发给 User
- 适合场景：Manager 有高度自治权，但 User 不完全信任它们

模式 4：自治子系统集群（Autonomous Subsystem Cluster）

```
User ──→ Planner/Orchestrator
                  │
        ┌─────────┼─────────┐
        ▼         ▼         ▼
    ┌─────────┐ ┌─────────┐ ┌─────────┐
    │ Sub-sys │ │ Sub-sys │ │ Sub-sys │
    │    A    │ │    B    │ │    C    │
    │┌───────┐│ │┌───────┐│ │┌───────┐│
    ││ Plan  │ │ │ Plan  │ │ │ Plan  │
    ││ Exec  │ │ │ Exec  │ │ │ Exec  │
    │└───────┘│ │└───────┘│ │└───────┘│
    └─────────┘ └─────────┘ └─────────┘
        │           │           │
        └───────────┼───────────┘
                    ▼
              汇总 → User
```

消息流： User 发出任务 → Planner/Orchestrator 拆解 → 分发给多个**自治子系统** → 各子系统内部独立闭环处理 → 结果汇总回 User。

核心特征： 子系统内部有自己的 Planner、Executor，甚至自己的策略。Orchestrator 只负责"分活"和"收活"，不干预子系统内部逻辑。

User 位置： 系统入口 + 最终汇总接收端。

控制逻辑变体（这是分类的关键）：

控制强度	机制	对应制度
无控制	Orchestrator 只是协商总线，子系统可以拒绝配合	周代分封、西欧封建、神圣罗马帝国
心跳控制	子系统自治，但必须定期向中央回传状态	江户幕藩（参觐交代）
外部API式	子系统是外部服务，只按接口协议返回结果	阿兹特克（附属城邦）

总计：5 个制度（周代分封、西欧封建、神圣罗马、江户幕藩、阿兹特克）

从 MAS 角度看，"弱中央"和"强中央"不是两种架构。 它们是同一种架构（自治子系统集群），区别只是 Orchestrator 的权限配置不同：

```
class Orchestrator:
    control_level: str  # "none" | "heartbeat" | "command"
    # "none"      → 神圣罗马帝国（子系统可以忽略 Orchestrator）
    # "heartbeat" → 江户幕藩（子系统必须定期汇报，否则触发告警）
    # "command"   → 不存在于这些制度中（那就退化成模式 1 了）
```

MAS 实现要点：

- 每个子系统是一个独立的 Agent（或 Agent Group），有自己的 System Prompt 和工具集
- Orchestrator 需要处理：任务拆分、结果汇总、冲突调解
- 子系统之间**是否能直接通信**是一个关键设计决策（这些历史制度中大多不允许，都必须通过中央）
- 心跳机制的实现：定时器 + 子系统必须响应 `health_check()` 调用

模式 5：多级门控管道（Multi-Gate Pipeline）

```
Initiator ──→ Planner ──→ Gate1 ──→ Gate2 ──→ Executor ──→ Auditor
                          (审议)    (签署)                   (追溯审计)
                            |         |                        |
                          可修改    可Veto                   可判无效
```

消息流： 任务经过多个独立的"门控（Gate）"节点，每个门控都有权力**修改、阻断或回滚**流程。是模式 2 的多级扩展版本。

User 位置：这是最复杂的部分——没有单一 User。

在前面所有模式中，User 都是一个明确的节点。但在模式 5 中，"User 的职能"被拆散到了多个节点：

User 的职能	在模式 5 中由谁承担	具体例子
发起任务	Initiator	议员提出法案、首相提出政策
审批方案	Gate（签署节点）	总统签署/否决、保民官 Veto
最终价值判定	Auditor	最高法院违宪审查、欧洲法院
启动系统	Symbolic Trigger	英国君主（仅提供合法性 Token）

所以这个模式的 User 在哪？答案是：到处都是，又不在任何一个地方。 如果强行指定一个 User，最合理的选择是把 **Initiator** 当作 User（因为任务从它开始），但它的权力远小于模式 1-4

中的 User。

对应制度：

制度	Initiator	Gate 节点	Auditor
美国联邦	议员	国会委员会 → 参众两院 → 总统	最高法院
英国议会君主制	首相/内阁	上院 → 下院	无独立追溯审计
罗马共和国	执政官(×2)	元老院 → 人民大会 → 保民官(Veto)	无
欧盟	欧盟委员会	欧洲议会 + 部长理事会(Co-decision)	欧洲法院

总计：4 个制度

MAS 实现要点：

- 每个 Gate 是一个独立 Agent，有自己的判断标准
- 需要定义 Gate 的行为类型：`modify`（修改后放行）、`approve/reject`（二元判定）、`veto`（一键全局中断）
- Auditor 是**后置的**——它不阻止执行，但可以在执行完成后判定结果无效
- 最大的实现挑战是**状态管理**：一个任务在多个 Gate 之间流转时，每个 Gate 可能修改任务内容，需要保持版本追踪

模式 6：共识驱动型（Consensus-Driven）

```
        ┌──────────────────────┐
        │   Planner（可随机）    │
        └──────────────────────┘
                   │ 提案
                   ▼
    ┌──────────────────────────┐
    │   Consensus Layer（投票/协商）  │  ← 这就是 User
    │   [Agent1] [Agent2] ... [N]  │
    │   Vote / Debate / Unanimity  │
    └──────────────────────────┘
                   │ 通过
                   ▼
              Executor(s)
```

消息流： Planner 生成方案 → 提交给一组 Agent 进行集体决策 → 通过后执行。

User 位置：User 就是那个 Consensus Layer。 不是单个节点，而是一组 Agent 的集体行为。决策方式可以是：

共识算法	对应制度
Majority Vote（多数决）	雅典公民大会
Binary Accept/Reject（只能整体通过或否决）	斯巴达公民大会
Unanimity（全员一致，一票否决）	易洛魁联盟大议事会
Weighted Consensus（基于威望的加权）	原始部落氏族议事会

对应制度：

制度	Planner 特点	共识机制	附加机制
雅典民主	随机抽签产生（防操控）	多数投票	陪审法庭追溯审计
斯巴达混合	终身长老（稳定性）	二元表决	督政官实时监控 + 双王热备
原始部落	氏族议事会协商	加权共识	无
易洛魁联盟	部落→联盟两级	全员一致	氏族母亲控制节点生命周期

总计：4 个制度

MAS 实现要点：

- Consensus Layer 的实现本质上是**多个 LLM 实例投票**
- 需要定义投票协议：每个 Agent 输出 `{vote: "approve"/"reject", reason: "..."}` → 汇总 → 按规则判定
- Unanimity 模式在节点数多时几乎无法通过 → 需要定义"妥协机制"或超时规则
- 随机化 Planner（雅典模式）：每次从模型池中随机抽取一个实例来生成方案，增加多样性

第二部分：六种模式总览

模式	名称	User 位置	核心机制	制度数量	代表制度
1	单链管道	入口（顶部）	单向流，无控制	8	**古埃及法老制**
2	审核回路管道	入口（顶部）	Reviewer 可 reject 回 Planner	1	**唐代三省六部**
3	旁路监控管道	入口 + 异常报告接收端	Monitor 旁路采样，不阻断主流	2	**波斯行省制**
4	自治子系统集群	入口 + 汇总接收端	子系统内部闭环，Orchestrator 分发/收集	5	**江户幕藩体制**
5	多级门控管道	**拆散到多个节点**	多个 Gate 可修改/阻断/回滚	4	**美国联邦**
6	共识驱动	**Consensus Layer 本身**	集体投票/协商决策	4	**雅典民主**

28 制度 = 8 + 1 + 2 + 5 + 4 + 4 = 24？

还剩 4 个制度没有被分入上面的模式。它们是：

制度	原因
蒙古札撒制	核心特征不是消息流模式，而是**执行层的递归结构**（万→千→百→十）。消息流本身是模式 1（单链管道），但 Executor 层的实现方式是递归的。
印加米塔制	类似蒙古的递归 + **全局共享状态（Quipu）**。这不是消息流模式的差异，而是**状态管理方式**的差异。
苏美尔城邦	核心特征是**应急模式切换**（公民大会 = Emergency Handler）。消息流本身是模式 1，但存在一个休眠节点可以在紧急情况下改变系统运行模式。
马里帝国	核心特征是**底层宪法硬护栏（曼丁卡宪章）+ 世袭专业化 Executor**。消息流本身是模式 1，但有不可违反的系统级约束。

这 4 个制度的特殊性不在于"消息流拓扑"不同，而在于"节点实现方式"或"系统级配置"不同。 它们可以在模式 1（单链管道）的基础上，通过以下方式扩展：

制度	基础模式	扩展特性	实现方式
蒙古	模式 1	递归 Executor	`Executor` 内部递归实例化子 `Executor`，每级 1:10
印加	模式 1	共享状态	所有 Executor 读写同一个 `SharedState` 对象
苏美尔	模式 1	应急中断	注册一个 `EmergencyHandler`，触发时重置所有节点优先级
马里	模式 1	硬护栏	所有节点输出经过 `SystemProtocol.validate()` 检查

所以最终的答案是：**6 种消息流模式 + 4 种节点级扩展特性**，覆盖全部 28 个制度。

第三部分：User 在哪？—— 完整 User 位置图谱

这是整份报告最关键的一张表。

User 的五种存在形态

从 MAS 角度，User 并不总是"系统入口的那个人类"。User 的本质是：**谁定义了系统的目标函数？谁有权终止系统？**

形态	含义	对应模式	对应制度
入口User	User 在系统入口，发出Prompt，等结果	模式1, 2, 3	古埃及、唐三省六部、波斯、明清、苏联、法国、罗马帝国、圣统制、哈里发、奥斯曼
入口+监听User	User 在入口发 Prompt，同时从 Monitor 接收异常报告	模式3	波斯、秦汉
分散User	User 的职能被拆散到多个节点，没有单一 User	模式5	美国、英国、罗马共和、欧盟
集体User	User 不是个体而是群体，通过投票/共识做决策	模式6	雅典、斯巴达、原始部落、易洛魁
委托User	User 把全部执行权委托给代理 Planner，只做最终验收	模式1变体	奥斯曼（苏丹委托大维齐尔）

User 接入点的代码级定义

```python
# 模式 1/2/3: User 是系统的 entry point
class Pipeline:
    def run(self, user_prompt: str) -> str:
        plan = self.planner.generate(user_prompt)
        # ... 审核/监控 ...
        result = self.executor.execute(plan)
        return result  # 返回给 User

# 模式 4: User 发起 + 接收汇总
class ClusterSystem:
    def run(self, user_prompt: str) -> str:
        sub_tasks = self.orchestrator.split(user_prompt)
        results = [sub_sys.run(task) for sub_sys, task in zip(self.subsystems,
sub_tasks)]
        return self.orchestrator.aggregate(results)  # 汇总返回给 User

# 模式 5: User 的职能被拆散
class MultiGatePipeline:
    def run(self):
        proposal = self.initiator.propose()        # "User" 的发起权
        plan = self.planner.refine(proposal)
        for gate in self.gates:
            plan = gate.review(plan)                # "User" 的审批权
            if gate.vetoed:
                return "VETOED"
        result = self.executor.execute(plan)
        audit = self.auditor.check(result)          # "User" 的追溯权
        if audit.invalid:
            return "NULL_AND_VOID"
        return result
    # 问题: 谁是 User? initiator? gates? auditor? 都是, 也都不是。

# 模式 6: User 是集体
class ConsensusSystem:
    def run(self):
        proposal = self.planner.generate()
        votes = [agent.vote(proposal) for agent in self.consensus_agents]  # 集
体投票
        if self.consensus_rule(votes):  # majority / unanimity / weighted
            return self.executor.execute(proposal)
        else:
            return "REJECTED"
    # User = consensus_agents 的集合
```

第四部分：MAS 构建建议

建议 1：只需要实现一个框架

不需要 6 套代码。所有模式可以用**同一个框架**表达：

```
通用框架 = 节点(Node) + 边(Edge) + 路由规则(Router)
```

每个 Node 有：

- `type`: planner / reviewer / gate / executor / monitor / consensus
- `enabled`: true / false
- `permissions`: [reject, veto, modify, read_only, none]
- `llm_config`: 用哪个模型、什么 System Prompt

每个 Edge 有：

- `from` / `to`: 源节点 / 目标节点
- `type`: command / submit / reject / approve / veto / heartbeat / broadcast
- `condition`: 触发条件（可选）

模式 1 = 所有 Edge 都是 `command` 类型，无 `reject` Edge 模式 2 = 加一条 `reject` Edge 从 Reviewer 回到 Planner 模式 3 = 加一个 `monitor` 类型的 Node，Edge 是 `read_only` 模式 4 = Node 可以是 `SubSystem` （内部包含自己的 Node/Edge 图） 模式 5 = 多个 `gate` 类型的 Node 串联 模式 6 = 一个 `consensus` 类型的 Node（内部包含多个投票 Agent）

建议 2：最小实现集合

如果优先级有限，只实现以下 4 个制度即可覆盖所有核心机制：

优先级	制度	模式	覆盖的机制
P0	**古埃及法老制**	1	基线：单链管道。框架的骨架。
P1	**唐代三省六部**	2	增加：审核回路（reject loop）
P2	**江户幕藩体制**	4	增加：自治子系统 + 心跳
P3	**美国联邦**	5	增加：多级门控 + 追溯审计

为什么不选模式 3 和 6?

- 模式 3（旁路监控）可以通过在模式 1 上加一个 `monitor` 节点实现，不需要单独建一个制度
- 模式 6（共识驱动）在技术上是模式 5 的特例——把 Gate 替换为 Consensus Layer 即可

建议 3：实验设计

最有价值的实验是给**同一个任务**分别用不同模式跑，比较：

指标	模式 1	模式 2	模式 4	模式 5
完成速度	最快	较慢（审核延迟）	取决于子系统	最慢（多级门控）
错误率	最高	较低	取决于子系统	最低
Token 消耗	最低	中等（审核轮次）	高（多子系统）	最高（多 Gate）
适用场景	简单确定性任务	需要质量保证	复杂可拆分任务	高风险低容错

建议 4：关于"不好弄"的具体拆解

你说"看着不太好弄"，具体的难点在于：

难点 1：Planner 怎么拆分任务？ 在模式 4（自治子系统）中，Planner/Orchestrator 需要把一个任务拆成多个子任务分发给不同的子系统。这需要 Planner 理解每个子系统的能力边界。 → 解法：给 Orchestrator 一个"子系统能力注册表"，让它根据任务类型做路由。

难点 2：多 Agent 之间的状态如何传递？ 当消息从 Planner 流向 Reviewer 再流向 Executor，每一步的输出都是下一步的输入。如何保持上下文？ → 解法：维护一个全局 `TaskContext` 对象，每个节点处理后往里面追加信息，下一个节点读取。

难点 3：Reviewer/Gate 怎么判断"合格"？ Reviewer 需要有自己的标准来决定 approve 还是 reject，这个标准怎么定义？ → 解法：Reviewer 的 System Prompt 里写明判断标准（如：检查计划是否完整、是否有安全风险等），让 LLM 自行判断。

难点 4：共识怎么达成？ 模式 6 中多个 Agent 投票，如果永远无法达成一致怎么办？ → 解法：设置超时机制 + 降级策略（比如 3 轮 Unanimity 不通过则降级为 Majority Vote）。

第五部分：缺失制度确认

我覆盖了 28 个制度。你说文档中有 30 个。请确认遗漏的 2 个是什么。

已覆盖清单：

1. 古埃及法老制
2. 罗马帝国元首制与君主制
3. 中国明清内阁制与军机处
4. 罗马天主教会圣统制
5. 阿拉伯-伊斯兰哈里发制
6. 法国绝对君主制度
7. 波斯阿契美尼德行省制
8. 秦汉郡县制
9. 唐代三省六部制
10. 中国周代分封制
11. 西欧封建领主制
12. 神圣罗马帝国
13. 日本江户幕藩体制
14. 阿兹特克三邦同盟
15. 雅典直接民主制
16. 古斯巴达寡头与民主混合制
17. 罗马共和国混合立宪制
18. 英国议会君主制
19. 美国联邦总统制
20. 蒙古帝国札撒与汗国制
21. 印加帝国米塔劳役制
22. 原始部落酋邦制
23. 易洛魁联盟
24. 苏美尔城邦制
25. 马里帝国曼丁卡治理制
26. 奥斯曼帝国御前会议制
27. 苏联共产党-国家体制
28. 欧洲联盟超国家治理体制